



EE214 Digital Circuits Laboratory

Wadhvani Electronics Laboratory
Electrical Engineering IIT Bombay

Problem set: 8

Date: October 1, 2026

UART Programmable Timer

Instructions:

1. Use **Behavioral** modeling for writing VHDL description.
2. Write the testbench to perform RTL simulation.
3. Demonstrate the simulations to your TA.
4. Perform Pin-Planning, run on Xenon board and demonstrate to your TA.
5. Submit the entire project files in .zip format in moodle.

Part I - Overview and System Design

In this experiment, you will design a programmable countdown timer using a Finite State Machine (FSM). The timer delay will be set dynamically via UART by entering the desired delay time in seconds (up to 255 seconds) using a terminal like TeraTerm or RealTerm.

Since the maximum display capability of our 8 LEDs is 255 (which is $2^8 - 1$), the timer supports delays up to 255 seconds. If a user enters a value greater than 255, the FSM must reject it and not start the timer.

FSM Specifications

You must implement a Finite State Machine consisting of the following states to capture the UART inputs and control the timer:

- **DIGIT_1 State:** The default state. The system waits for the first ASCII numeric input ("0" to "9") from the UART RX. Once received, it stores the digit and transitions to the **DIGIT_2** state. (If 'Enter' is pressed here instead, it should transition to **PROCEED** to handle 1-digit numbers).
- **DIGIT_2 State:** Waits for the second digit. Once received, it updates the accumulated value and transitions to the **DIGIT_3** state. (If 'Enter' is pressed, it transitions to **PROCEED** to handle 2-digit numbers).
- **DIGIT_3 State:** Waits for the third and final digit. Updates the accumulated value. Once 'Enter' (ASCII Carriage Return x"0D") is pressed, it transitions to the **PROCEED** state. If they are entering more than 3 digits its considered as greater than 255.
- **PROCEED State:** The FSM validates the accumulated 3-digit number.
 - If the value is ≤ 255 , it begins the countdown using a 1 Hz clock, decrementing the timer by 1 every second. The current remaining time must be displayed in binary on the 8 LEDs. When the counter reaches 0, transition to the **DONE** state.
 - If the value is > 255 , the input is rejected, the accumulator resets, and the FSM immediately returns to the **DIGIT_1** state.
- **DONE State:** The timer has finished. All 8 LEDs should continuously toggle ON and OFF (e.g., at 5 Hz) indefinitely until the system is reset.

Part II - Clock Divider

You will need to generate three different clock frequencies from the 50 MHz master clock:

1. **5 MHz Clock:** Used for the UART Baud Rate generator (9600 baud).
2. **1 Hz Clock:** Used for the 1-second countdown decrement.
3. **5 Hz Clock:** Used to create the continuous blinking effect on the LEDs when the timer finishes.

VHDL Description

The code below is your Top-level template. Complete the FSM logic and the counter processes.

```
1  library ieee;
2  use ieee.std_logic_1164.ALL;
3  use ieee.numeric_std.all;
4
5  entity UART_Timer is
6      generic (
7          g_CLKS_PER_BIT : integer := 521    -- For 5 MHz clock and 9600 baud
8      );
9      port (
10         i_Clk          : in  std_logic;    -- 50 MHz input clock
11         i_Reset         : in  std_logic;    -- Hardware Reset button
12         i_RX_Serial     : in  std_logic;
13         o_LED           : out std_logic_vector(7 downto 0)
14     );
15 end UART_Timer;
16
17 architecture rtl of UART_Timer is
18
19     -- Clocks
20     signal clk_5MHz : std_logic;
21     signal clk_1Hz  : std_logic;
22     signal clk_5Hz  : std_logic;
23
24     -- UART Signals
25     type t_SM_Main is (s_Idle, s_RX_Start_Bit, s_RX_Data_Bits,
26                        s_RX_Stop_Bit, s_Cleanup);
27     signal r_SM_Main : t_SM_Main := s_Idle;
28     signal r_RX_Data_R : std_logic := '0';
29     signal r_RX_Data : std_logic := '0';
30     signal r_Clk_Count : integer range 0 to g_CLKS_PER_BIT-1 := 0;
31     signal r_Bit_Index : integer range 0 to 7 := 0;
32     signal r_RX_Byte : std_logic_vector(7 downto 0) := (others => '0');
33     signal r_RX_DV : std_logic := '0';
34
35     -- FSM States for Timer
36     type t_Timer_State is (S_DIGIT_1, S_DIGIT_2, S_DIGIT_3, S_PROCEED, S_DONE);
37     signal r_Timer_State : t_Timer_State := S_DIGIT_1;
38
39     signal r_Accumulator : integer := 0;
40     signal r_Timer_Count : integer range 0 to 255 := 0;
41     signal r_Blink_State : std_logic := '0';
42
43     -- Clock Divider Component
44     component ClockDiv is
45         port (
46             clk_in      : in  std_logic;
47             reset       : in  std_logic;
48             clk_out_5MHz : out std_logic;
49             clk_out_1Hz  : out std_logic;
50             clk_out_5Hz  : out std_logic
51         );
52     end component;
```

```
53
54 begin
55
56 -- Add your code here
57 end rtl;
```

On Xenon board

- Map the 50 MHz clock to your input `i_Clk`.
- Map an onboard push button to `i_Reset`.
- Pin Plan for UART Rx using the same UART GPIO mapping as the previous lab.
- Map `o_LED` to LEDs on the Xenon board.
- Connect Xenon via USB to the PC. Open your Serial Terminal (9600 baud).
- **Testing:**
 - Type 10 and press **Enter**. Verify the LEDs count down from 10 (in binary) and then blink rapidly.
 - Press Reset.
 - Type 300 and press **Enter**. Verify the LEDs remain OFF (FSM rejection logic working).